Praktikum 2 - Pemrograman Web (MVC)

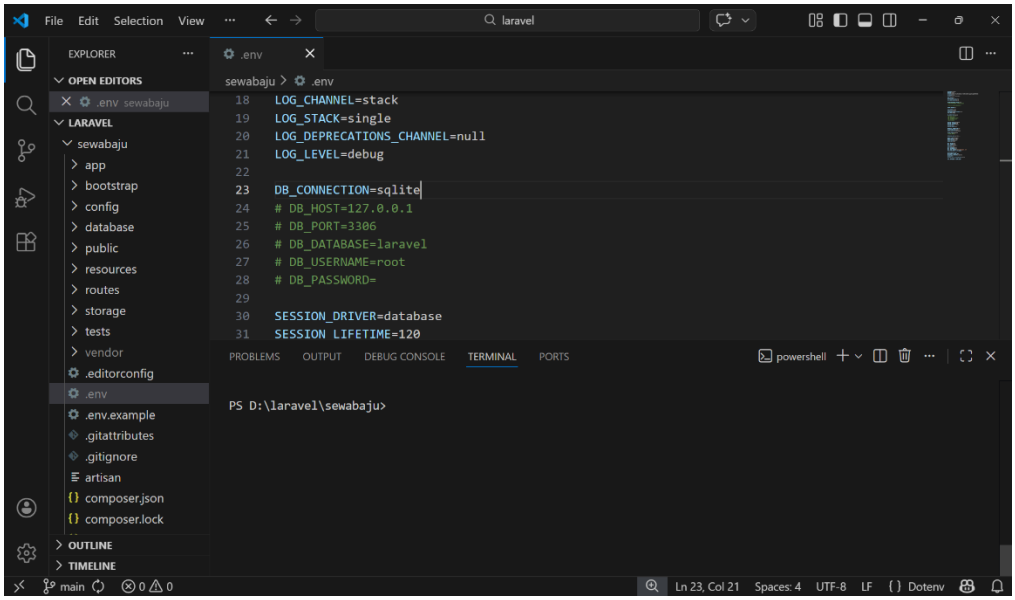
Lakukan praktikum dibawah ini, dan buat screenshot untuk pembuktian mengerjakan setiap poin dengan mengisi tabel dibawah, kemudian tunjukan hasil akhir dari men-share repository github yang telah dibuat.

Notes: Jika ada code XX maka ganti dengan 2 digit NIM terakhir anda, **agar mencegah bentrok** dengan tim lainya jika sudah masuk ke **github**. Ini hanya berlaku di praktikum untuk uji-coba, apabila di project PBL tidak perlu menggunakan XX.

A. Konfigurasi Database Mysql pada Laravel 12

Pada Laravel 12 terdapat default database ke sqlite, untuk itu ubah ke mysql dengan mengubah setting dengan langkah seperti dibawah ini:

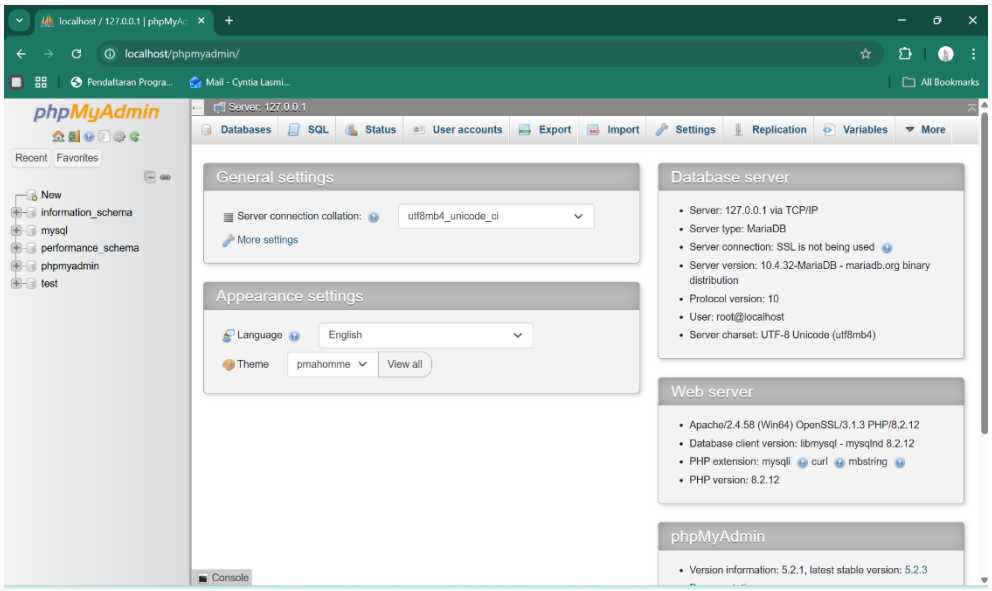
1. Buka File .env pada project laravel dan cari DB_CONNECTION



Note: **Ganti sqlite -> mysql dan aktifkan settingan baris dibawahnya, pada DB_DATABASE ubah nama db sesuai yang akan digunakan untuk project**

Hasil Screenshoot:

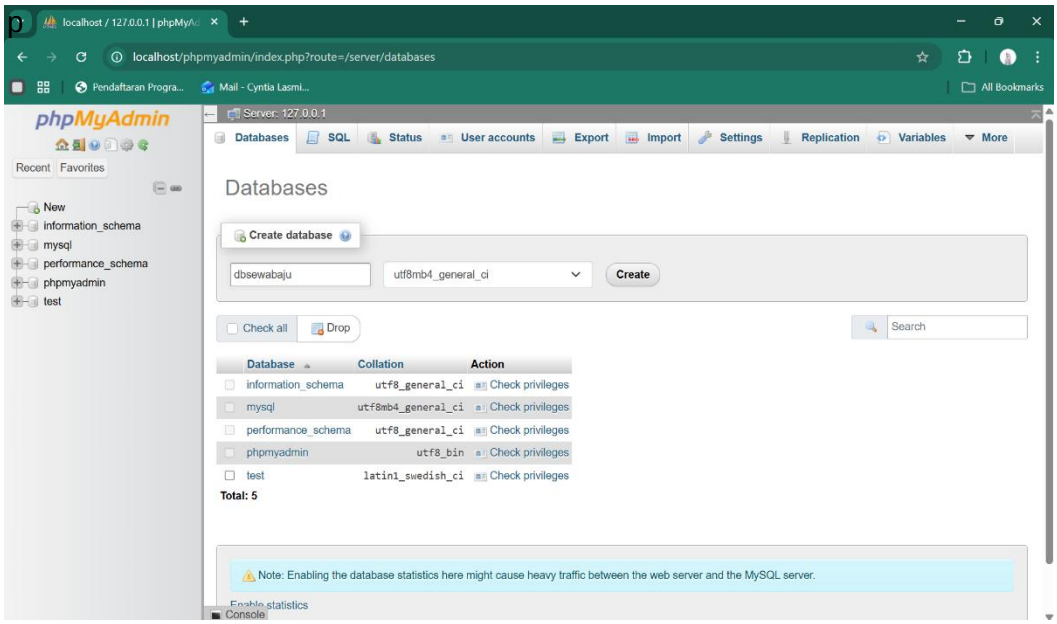
2. **Pada Xampp jalankan apache dan mysql kemudian gunakan browser menuju localhost/phpmyadmin, dan buat database baru dengan project anda (sesuaikan dengan yang anda buat diatas nama database)**



3. **Buat database baru dengan proyek anda sesuai instruksi nomor 2,**

Hasil Screenshoot:

4. Pastikan nama database sama seperti DB_DATABASE pada gambar diatas (disesuaikan dengan nama db yang akan dibuat, mis: DB_DATABASE=dbsewabaju)



5. **Pastikan juga user database dan password sesuai (Apabila password mysql belum pernah diganti, maka default DB_USERNAME = root, dan DB_PASSWORD kosong = Lihat di bagian Konfigurasi Database di Visual Studio Code)**
6. **Lakukan db migration untuk mensetup awal database pada laravel, (karena sebelumnya menggunakan sqlite)**
7. **Untuk melakukan migrasi bisa ketikan ini di terminal visual code**
   php artisan migrate

Hasil Screenshoot:

php artisan db:seed

Hasil Screenshoot:

8. Uji coba konesksi mysql dan laravel dengan menggunakan **php artisan serve**

```
PS D:\laravel\sewabaju> php artisan serve

  INFO  Server running on [http://127.0.0.1:8000].

  Press Ctrl+C to stop the server
```

9. Kemudian test di browser.

Hasil Screenshoot:

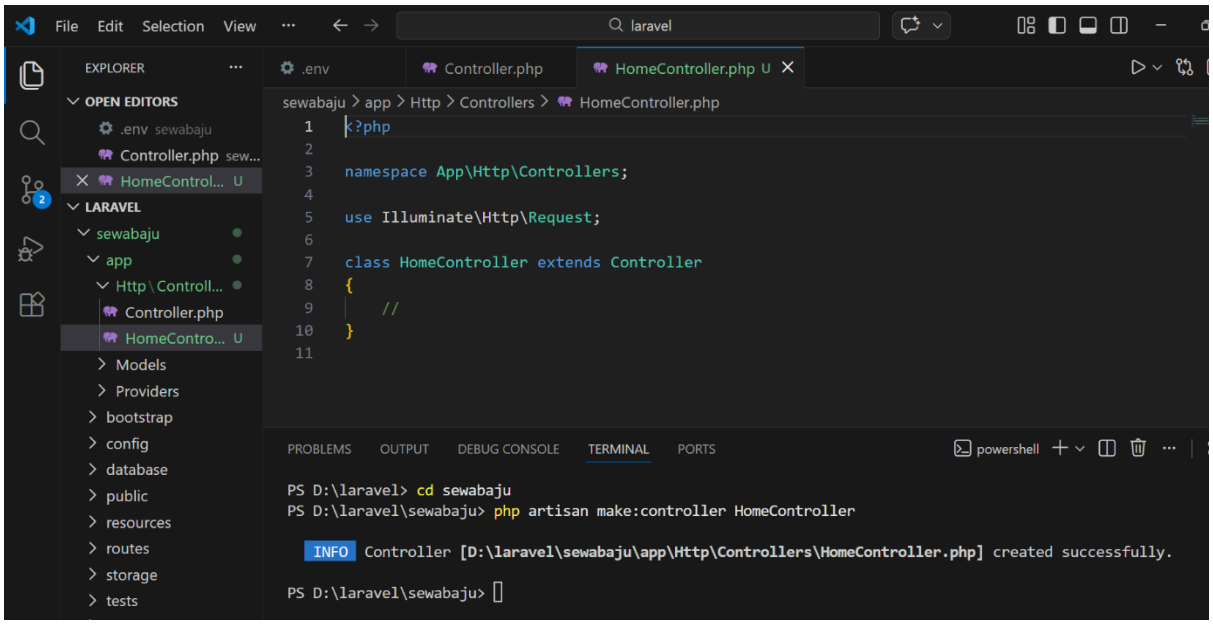
B. Simple Project Controller, Routes, Views

1. Buka terminal atau ketik (Ctrl+`)
2. Buat controller baru dengan mengetikan perintah berikut di terminal

php artisan make:controller HomeController

Hasil Screenshoot:

Maka akan dibuatkan controller baru, dan anda bisa melihat di app/Http/Controllers/HomeController



HomeController adalah nama controller, anda **bisa ganti dengan nama lainnya sesuai dengan kebutuhan project anda.**

Pada file homeController ketikan seperti gambar berikut

```php
      <?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class HomeController extends Controller
{
    public function index()
    {
        return view('home');
    }
    public function contact()


    {
        return view('contact');
    }
}
```

Hasil Screenshoot:

3. Buatlah Route untuk index dan contact, pada file routes/web.php, jadikan komentar untuk route sebelumnya pada view('welcome) seperti pada gambar berikut
   Tambahkan juga pada bagian atas
   Use App\Http\Controllers\HomeController

```php
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\HomeController;

//Route::get('/', function () {
  //  return view('welcome');
//});

Route::get('/', [HomeController::class, 'index']);
Route::get('/contact', [HomeController::class, 'contact']);
```

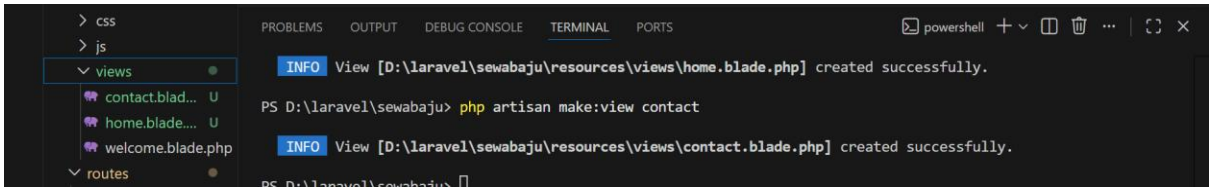
Hasil Screenshoot:

4. Buatlah view untuk home dan contact dengan menggunakan php artisan diTerminal
   php artisan make:view home

Hasil Screenshoot:

php artisan make:view contact

Hasil Screenshoot:



5. Buka File resources\view\home.blade.php dan resources\view\contact.blade.php. Ketikan **home.blade.php**: Selamat Datang di Toko Sewa Baju Terbaik (dikreasikan saja)

Hasil Screenshoot:

Ketikan di **contact.blade.php**: Silangkan menghubungi … (Dikreasikan saja)

Hasil Screenshoot:

6. Lakukan **php artisan serve,** lalu masukan link berikut di browser
http://localhost:8000/

Hasil Screenshoot:

http://localhost:8000/contact

Hasil Screenshoot:

E. Membuat variabel pada Controller dan mengirim kepada View
Ada dua metode untuk mengirim variabel dari controller ke view yaitu dengan **with** dan **compact.**

Dengan menggunakan With

- ➔ method chaining yang digunakan setelah fungsi `view()` untuk mengirim data satu per satu ke view dalam bentuk pasangan **key dan value**.

Pada HomeController.php ketik seperti pada gambar berikut:

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class HomeController extends Controller
{
    public function index()
    {
        $data = [
            'nama' => 'Budi',
            'pekerjaan' => 'Developer',
        ];
        return view('home')->with($data);
    }
    public function contact()
    {
        return view('contact');
    }
}
```

Mengirim array $data dengan isi data didalamnya adalah nama dan pekerjaan pada view home.

Lalu dalam **resource/view/home.blade.php**
Dan ketik seperti pada gambar berikut

```php
<div>
    <!-- Simplicity is an acquired taste. - Katharine Gerould -->
Selamat Datang di Toko Sewa Baju Terbaik {{$nama}}<br>
Anda adalah seorang {{$pekerjaan}}
</div>
```

Kemudian reload browser, di home (localhost:8000). Maka akan muncul variable sesuai dengan data array di controller.

Hasil Screenshoot:

Anda bisa juga menggunakan **compact** (Gunakan apabila data yg dikirimkan sedikit)

➔ fungsi bawaan PHP yang otomatis membuat array dari nama variabel yang disebutkan

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class HomeController extends Controller
{
    public function index()
    {
      //  $data = [
        //    'nama' => 'Budi',
          //  'pekerjaan' => 'Developer',
        //];
        // return view('home')->with($data);
        $nama = "Teddy";
        $pekerjaan = "programmer";
        return view('home', compact('nama', 'pekerjaan'));
    }
    public function contact()
    {
        return view('contact');
    }
}
```

Kemudian reload browser, di home (localhost:8000). Maka akan muncul variable sesuai dengan data array di controller.

Hasil Screenshoot:

Router mengatur url link anda

(Seo-friendly)	(tidak SEO friendly)
localhost/about	localhost/opt
localhost/admin/settings	localhost/set
localhost/artikel/makanan-khas-indonesia	localhost/artikel/aid=120230

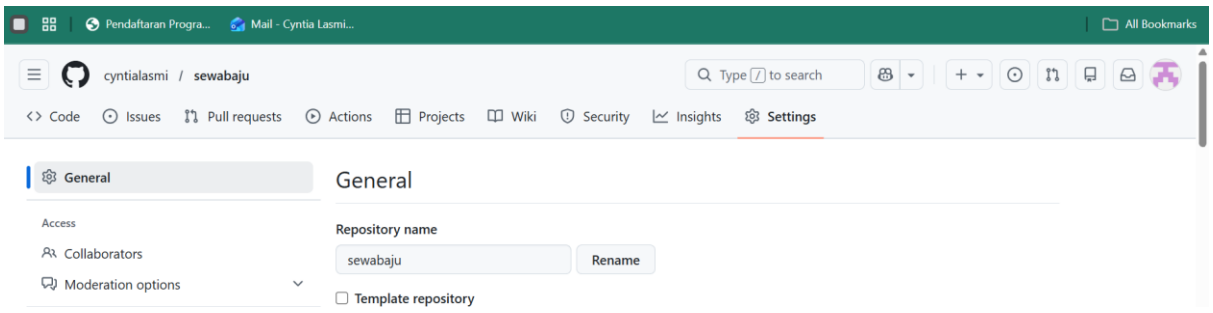
(Buat link SEO friendly apabila menginginkan kemudahan di cari di google)

Controller, menyimpan variabel, logika web, koneksi database dan lainnya

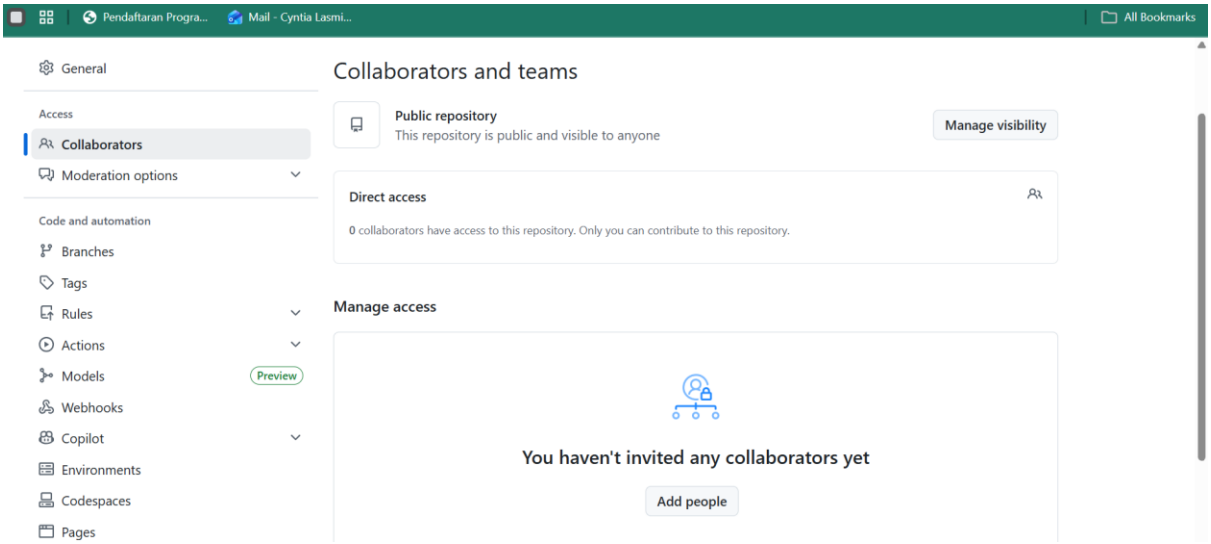
View, membuat tampilan website.

C. Setting Team Github

1. Buka akun github menggunakan browser
2. Pilih project anda, lalu pilih Settings > Collaborators (Perhatikan Gambar Dibawah)



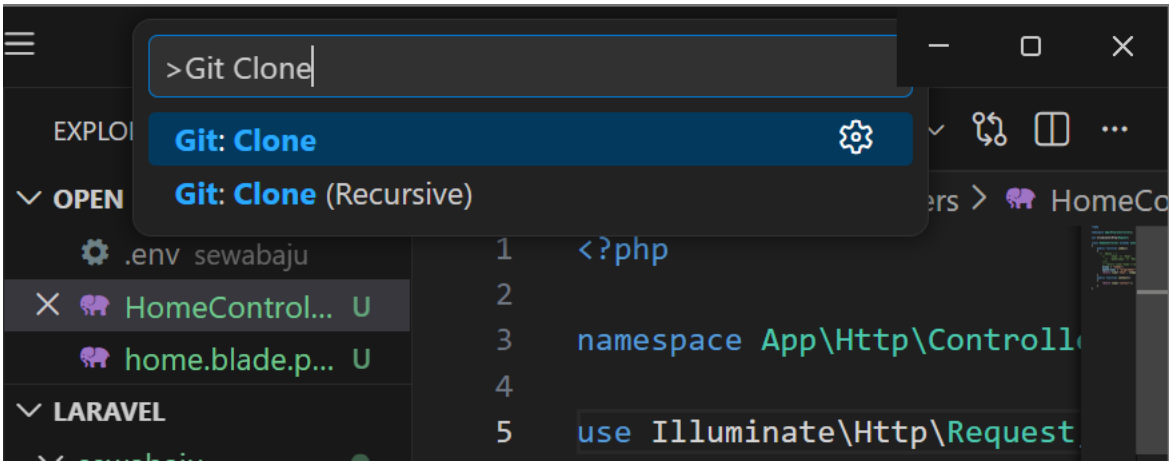
3. Tambahkan akun github tim anda di bagian “Add People”, masukan email/akun github



4. Setelah ditambahkan berarti team anda sudah bisa menggunakan project yang sama seperti anda.

B. Clone Github Repository

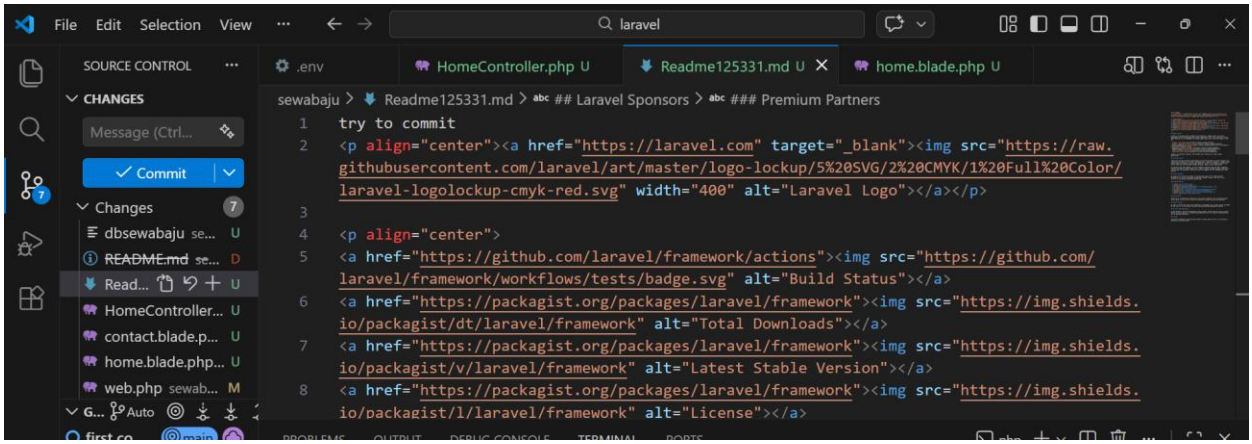
1. Buka Visual Code, pastikan sudah login github di visual code. Kemudian tekan (CTRL+Shift+P) untuk mengaktifkan command di atas, kemudian ketik, dan pilih > Git Clone



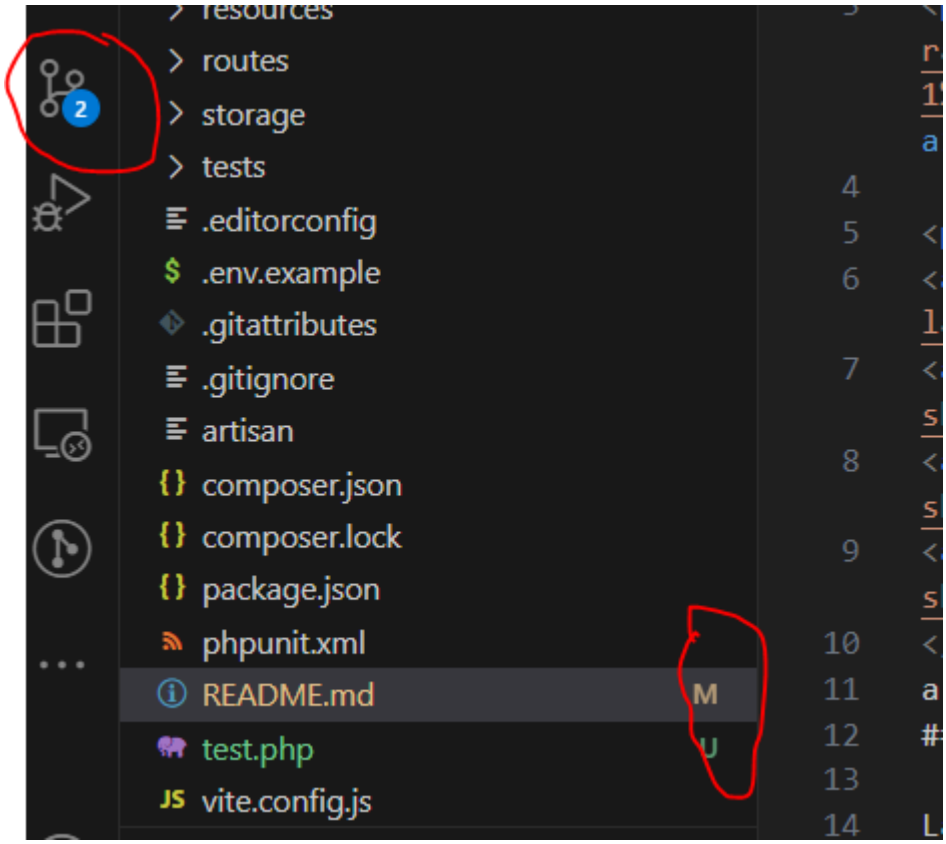
2. Pilih project Git dan simpan pada hardisk anda.

C. Mengubah/Menambah File

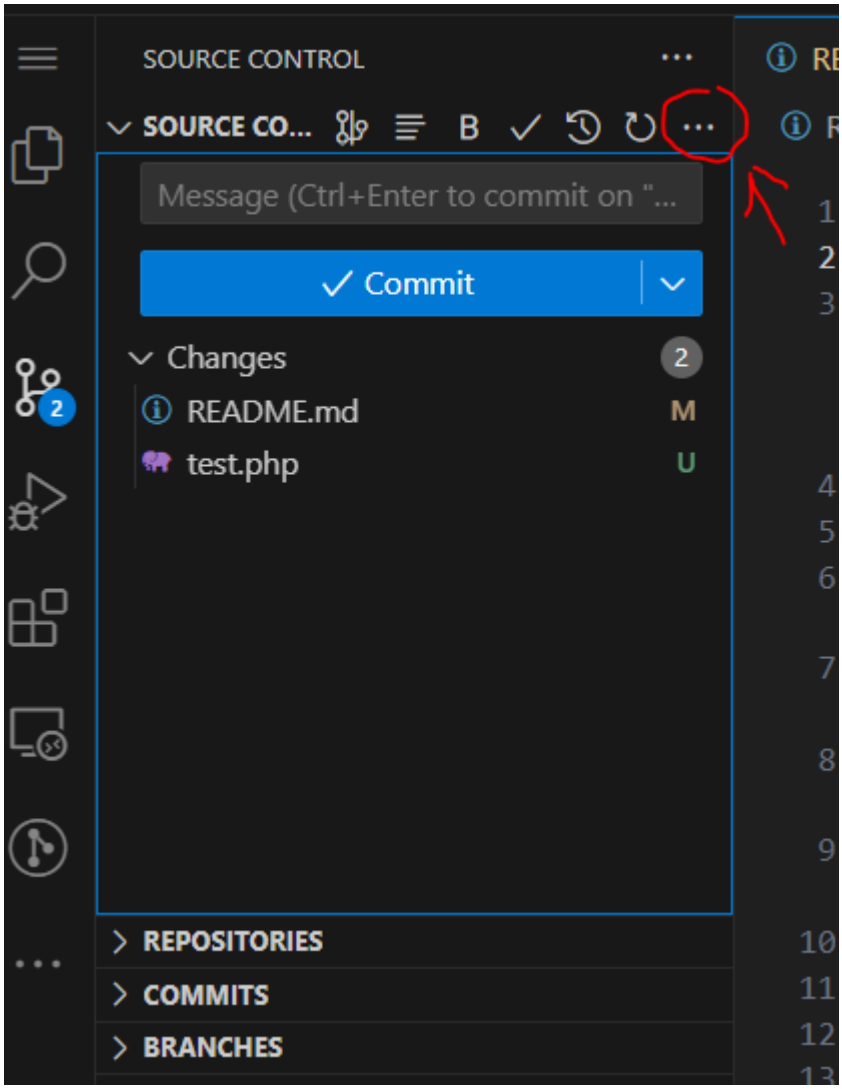
1. Pada project laravel yang telah di clone dari github, buka file Readme.md dan ubah nama file dengan ReadmeXX.md dan isi file tersebut dengan menuliskan kata apa saja di atas file tersebut. Lalu klik save.
2. Di dalam project, klik kanan make new file, Buatlah sebuah file dengan nama file dengan XX adalah dua digit nim terakhir anda testXX.php lalu masukan kata apa saja didalamnya



3. Perhatikan Huruf disebelah kanan file tersebut ada warna kuning “M” artinya Modified, dan U artinya “Update”
4. Untuk mengupdate kedalam repositori github, pilih icon fork di kiri visual code

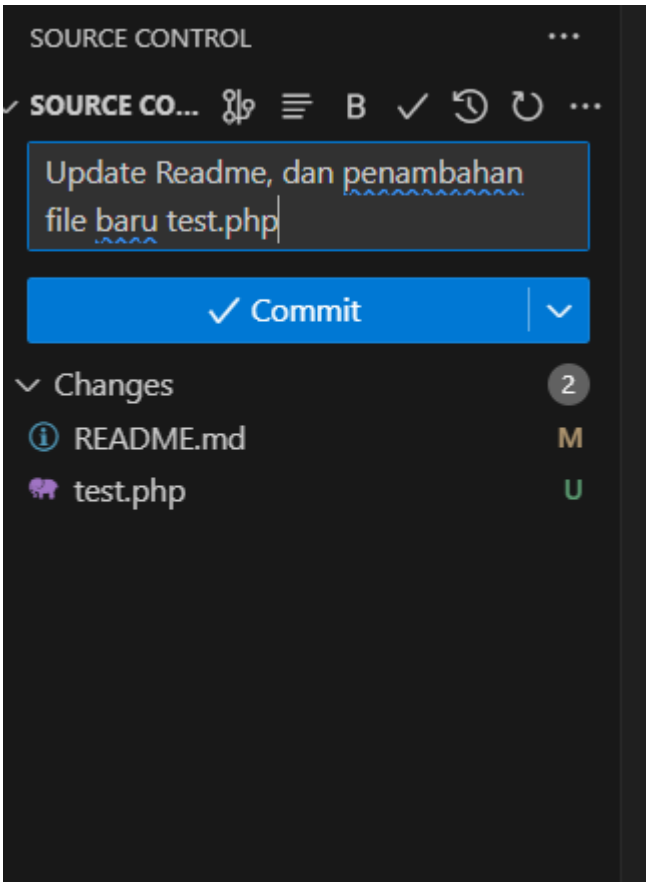


5. Jika dipilih maka akan menampilkan gambar berikut:



6. “Commit” artinya melakukan update ke repository github, **tapi perlu diperhatikan,** dalam melakukan team programming, kita **tidak boleh langsung saja melakukan commit.** Pastikan **melakukan Sync/Fetch** terlebih dahulu untuk memastikan ada tim lain yang melakukan perubahan terlebih dahulu.
7. Melakukan sync, klik “...” pada gambar yang dilingkari, lalu pilih **Fetch.**

8. Apabila sudah done fetch, maka kita lakukan commit dengan menambahkan pesan terlebih dahulu seperti pada gambar dibawah ini:

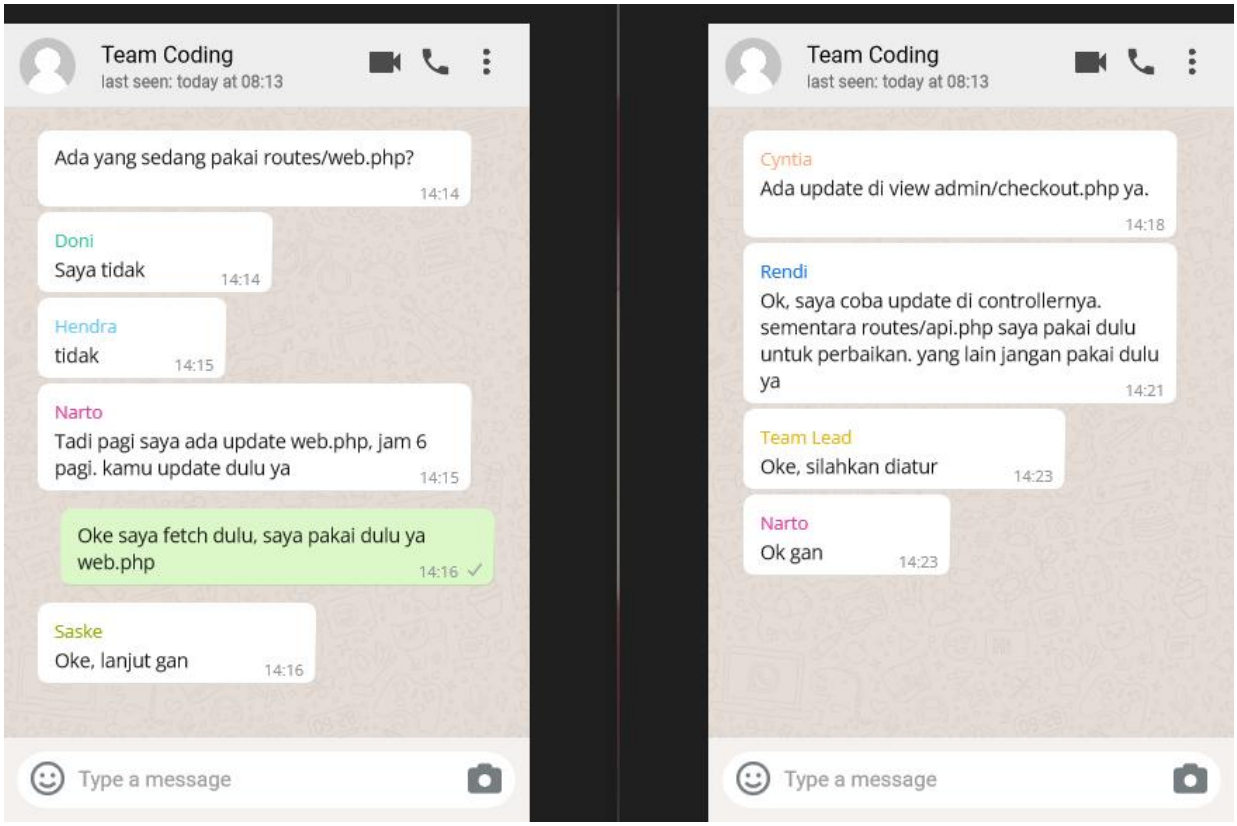


9. **Menulis pesan sebelum commit** adalah wajib, dalam Visual Code apabila tidak memasukan pesan, ada kemungkinan mengalami BUG dan visual code harus di close/open ulang.

Usahakan commit tidak boleh berbarengan dengan tim lain dengan **nama file** yang sama. Jadi pastikan setiap orang dalam tim memiliki filenya sendiri-sendiri karena akan menyebabkan konflik.

1. Tim akan menghandle file yang berbeda-beda seperti bagian view, model atau bagian controller,
2. Tetapi ada saatnya perlu update file yang sama seperti file config, routes, dan lainnya.
3. Pastikan komunikasi tim terjaga apabila ingin mengambil sebuah file untuk di update, pastikan menanyakan ke tim, apakah file tersebut dipakai.
4. Apabila sudah melakukan update sebuah file dan melakukan commit, maka infokan kepada tim ada perubahan atau ada file yang sudah diupdate dan berikan keterangan file apa yang sudah diupdate.
5. Pastikan file yang di commit itu sudah oke/teruji **tidak** menggantung **(belum selesai)** dan tidak menyebabkan error. File yang belum selesai, biarkan pada komputer lokal masing-masing programmer.

Selain teknis menggunakan Visual Code, komunikasi tim juga penting dalam bekerja menggunakan repository. Berikut contoh percakapan tim programming, yang biasanya terjadi pada dunia kerja.



Pada percakapan diatas, ada kesalahan yang dilakukan Narto yaitu, dia pernah update/commit web.php namun tidak di infokan ke group. Untungnya tim lain menanyakan terlebih dahulu. Jika Narto tidak menginformasikan, maka yang akan merugi adalah narto sendiri, karena nanti file web.php akan tertimpa oleh tim yang lain, sedangkan pembaruan file yang dilakukan narto pukul 6 pagi akan terhapus.
Yang dilakukan Rendi sudah benar, karena melakukan pemberitahuan akan memakai file routes/api.php, hal ini harus dilakukan apabila anda ingin melakukan perbaikan di sebuah file, yang diperkirakan akan membutuhkan waktu pengerjaan yang lama.

D. Recovery Project Cloning Github

Notes: Apabila anda baru saja melakukan **cloning dari github** dari project teman anda di komputer lain, maka anda harus mengulang install composer di direktori anda yang baru. Karena konfigurasi **composer** tidak di upload ke **github**, dan setting **env** tidak update ke github maka yang akan terjadi pada project laravel anda adalah error.

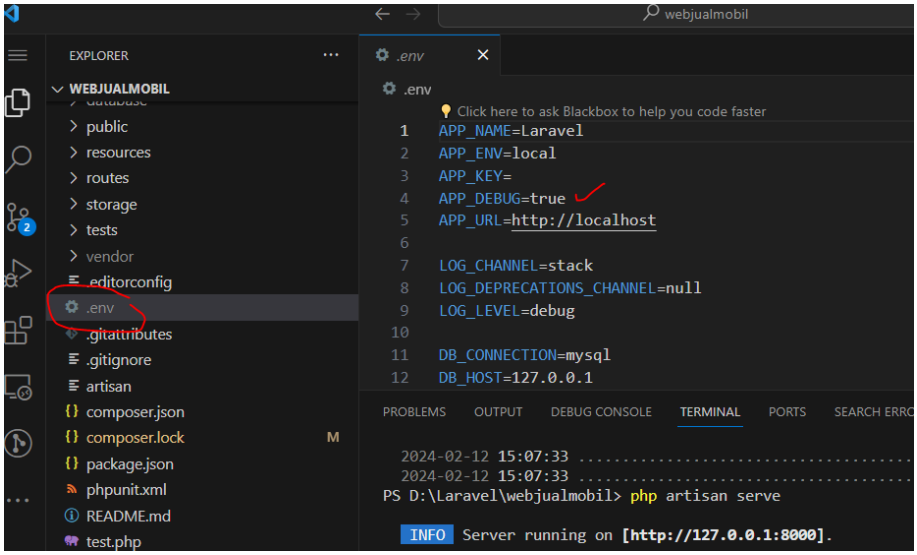
Apabila melakukan:
php artisan serve (akan menghasilkan error)

Maka anda harus melakukan ini di terminal, pada project laravel (CTRL+`)

composer install (tunggu hingga selesai)
composer dump-autoload (tunggu hingga selesai)
composer update (tunggu hingga selesai)

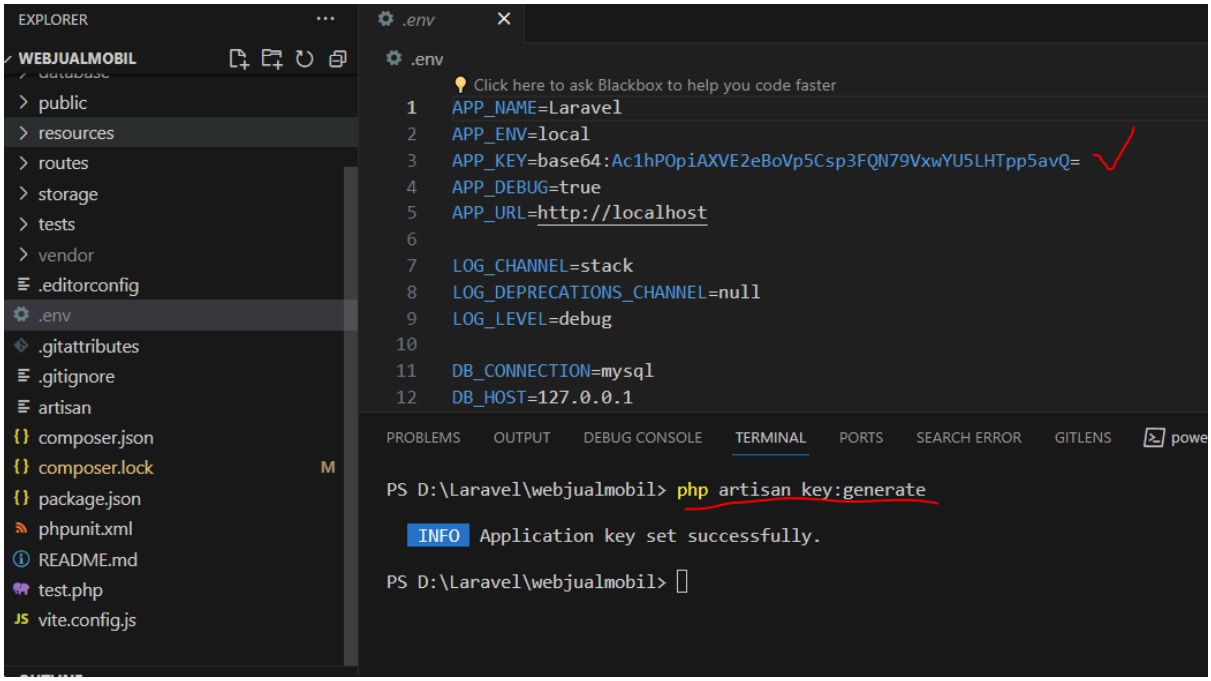
Setelah itu coba melakukan,
php artisan serve (kemudian check di browser)

Apabila pada browser menghasilkan Error Code 500. Maka harus dilakukan recovery laravel dengan mengubah file
.env-example menjadi **.env,** dan pastikan APP_DEBUG=true

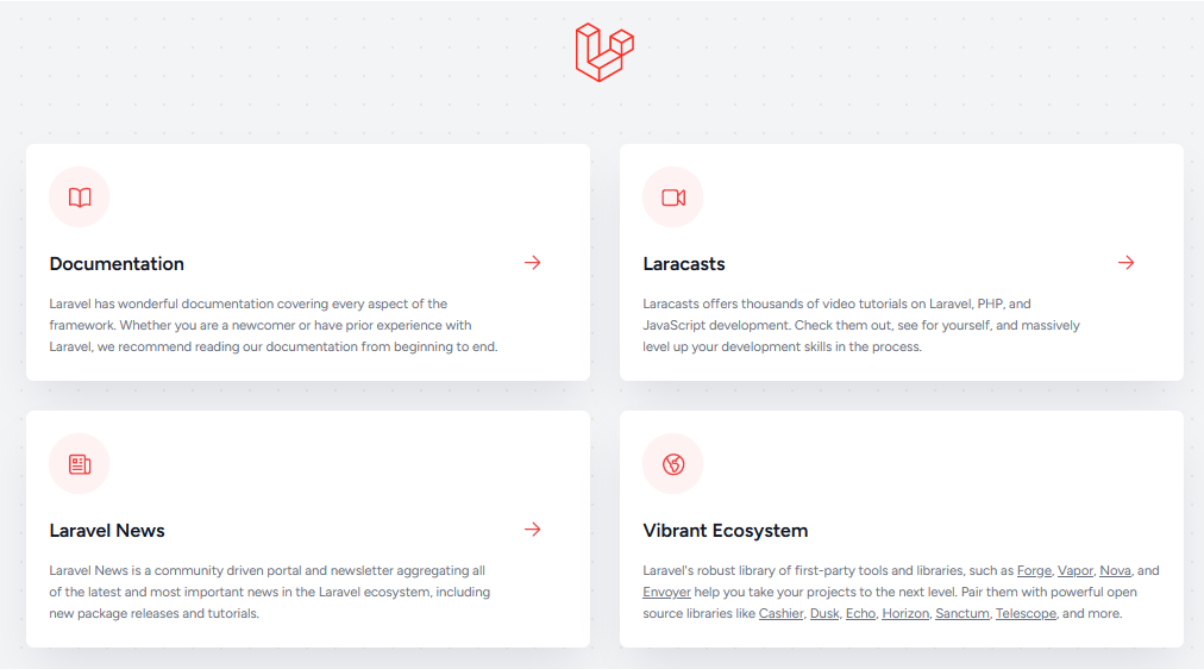


Kemudian membuat keysign baru, karena keysign tidak diupdate ke github. Pada terminal ketik berikut:

php artisan key:generate (maka APP key akan otomatis tergenerate



Setelah membuat key baru, maka anda sudah bisa menjalankan laravel, lakukan php artisan serve, lalu open dengan menggunakan browser.



Mengapa hal ini bisa terjadi?

Laravel mencegah data-data sensitif seperti database password, key sign, dan lainya diupload ke dalam github. Karena data-data tersebut akan berbahaya apabila terlihat oleh user lainnya. Semua folder/file yang di blokir ada di dalam list file **.gitignore.**

Tugas 2:

Semua hasil kerja screenshoot pada halaman ini, silahkan pindahkan ke template yang sudah disediakan. Jangan Lupa Tugas 1 Juga dilampirkan pada halaman awal tugas ke-2